

SD Basic - Printing

Print units, headings and page breaks, and sending output somewhere other than the screen.

Printing in SD goes through a **print unit** — a numbered destination that can be the terminal, a printer, a file or a spooler. This page covers directing output to one, and the paging that goes with it.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Print units

There are numbered print units. **Unit 0 is the terminal**. Any statement that produces output takes an `on` clause to choose a different one:

```
print on 1 'this goes to unit 1'
heading on 1 'so does this heading'
```

Without an `on` clause, output goes to the **current** unit — which is the terminal until `printer on` redirects it.

```
printer on
printer off
```

`printer on` sends `print` to the print unit; `printer off` sends it back to the terminal. `crt` ignores both and always writes to the screen — see SD Basic - Terminal Input and Output.

Choosing a destination

```
printer file {on print.unit} file.name, record.name {then ... else ...}
printer name {on print.unit} printer.name
printer display {on print.unit}
printer close {on print.unit}
printer reset
```

file	write to a record in a directory file
name	send to a named Windows printer
display	send to the terminal
close	finish the job and release the unit
reset	put every unit back to its default

The unit number goes after `on`, not first. `printer file on 1 'REPORTS', 'DAILY'` is right; `printer file 1, 'REPORTS', 'DAILY'` is a compile error, because the parser reads the `1` as the file name and then wants a comma it does not find.

A PRINT JOB END TO END

```
printer file on 1 'ZZOUT', 'REPORT'
heading on 1 "REPORT PAGE 'PL'"
printer on
print on 1 'first line'
print on 1 'second line'
printer close on 1
printer off
```

writes a record of **4 lines**, the first of which is `REPORT PAGE 1` — the heading, with `'PL'` replaced by the page number, followed by the two data lines and the blank the heading leaves under itself.

`printer close` **is what actually writes the file**. Until it runs the output is buffered in the unit. A program that ends without closing its units gets them closed for it, but a program that opens a second job on the same unit without closing the first merges the two.

Headings, footings and page breaks

```
heading {no.eject} {on print.unit} text
footing {on print.unit} text
page {on print.unit} {page.number}
```

A heading is reprinted at the top of every page and a footing at the bottom. Setting either resets the page counter and starts a new page — **unless** `no.eject` is given, which is how you set a heading without wasting the page you are on.

The text may contain codes, written inside quotes within the string:

CODE

'PL' . 'P'	the page number
'D'	the date
'T'	the time
'N'	suppress the automatic page break
'L'	a line break within the heading
'C'	centre what follows

Because the codes are quoted inside the heading, the heading itself needs the other kind of quote around it:

```
heading on 1 "SALES BY REGION 'L' 'D' 'C' PAGE 'PL'"
```

page forces a break, optionally resetting the number.

A heading is per print unit and it persists. It stays set until it is changed or the unit is reset, including after the program that set it has finished. A program that sets a heading on unit 0 and exits leaves the next thing the user does wearing that heading. `printer reset` **on the way out**, or set the heading on a unit you also close.

Asking about a unit

```
getpu(key, print.unit)
setpu key, print.unit, value
printer.setting(key, print.unit, value)
```

For unit 0 in a fresh session:

KEY	MEANING	VALUE
0	is the unit defined?	1
2	width in columns	80
3	length in lines	66
15	current page number	1

KEY	MEANING	VALUE
1002	lines left on this page	66

Other keys worth knowing:

KEY	MEANING
1	mode — where the output is going
4 · 5 · 6	top, bottom and left margins
7	spooler flags — landscape, duplex, raw mode and the rest
9	form name
12	number of copies
1003 · 1004 · 1005	heading, footing and data lines per page

`setpu` changes a setting; `printer.setting()` takes **three** arguments and reports or sets a spooler-level setting. **Not every key is valid for every call** — `printer.setting(0, 0, 0)` fails with *"Unrecognised printer setting key"*, because key 0 belongs to `getpu`, not to it.

The width and length are the unit's, not the terminal's. Unit 0 reports 80 by 66 in a session whose terminal has been set to 200 by 9999 — see `@crtwide` in SD Basic - Terminal Input and Output. A report that lays itself out from `getpu` and a screen that lays itself out from `@crtwide` are reading two different numbers, and both are right.

Capturing output instead of printing it

```
como on name
como off
```

`como` records everything the session displays into a record, which is the simplest way to keep a transcript of a run. To capture the output of one command rather than a whole session, `execute ... capturing` is better — see SD Basic - Program Control.

Suppressing output

```
hush on | off | expression
```

Discards output entirely rather than redirecting it. It applies to the session, not to a print unit, and it does not nest — the caution in SD Basic - Terminal Input and Output applies here too.

Printing on this port

A named printer is a Windows printer. `printer name` takes the name as Windows knows it, and the spooler flags in key `7` include a **raw mode** that sends bytes straight through without the driver reformatting them — which is what a line printer or a label printer needs.

There is no `lp` and no PostScript pipeline. The Linux original assumed both; this port hands the job to the Windows spooler instead. A report that worked by writing PostScript to a pipe needs a driver that accepts it, or raw mode and a printer that understands what you send.

What is not here

Nothing in the printing group has been removed from this port, but the **destination model changed**: printing goes to the Windows spooler rather than to a POSIX print command, and `printer.setting` keys that named a Linux facility do not apply.

See also

SD Basic - Terminal Input and Output · SD Basic - Data Conversion · SD Basic - Program Control.
